Manual técnico de iShop

Arquitectura, funcionamiento interno y operación del asistente con RAG

Proyecto de tesis iShop

22 de octubre de 2026

Índice

Alcance 1
Arquitectura 2
Topología de red 3
Estructura del repositorio 3
Recorrido completo de una consulta 4
El pipeline RAG 5
- Fuente del conocimiento 5
- Indexación 5
- Documentos cargados desde el formulario 6

El traspaso a un asesor humano 6
Configuración 7
Scripts 8
Operación 8
- Comandos frecuentes 8
- Actualizar el catálogo 8
- Respaldo 8
- Prueba automática 9

Fallas conocidas y cómo diagnosticarlas 9
- El bot no responde y n8n registra “webhook not registered” 9
- El bot responde vacío y el log dice “Invalid Qdrant URL” 9
- Se duplican los workflows al importarlos 9
- Un asesor entra a Chatwoot y no ve ninguna conversación 9
- Un usuario de n8n no ve los workflows 10
- La indexación tarda más de una hora 10

Decisiones de diseño y límites 10

Alcance

Este manual explica **cómo funciona el sistema por dentro**: qué hace cada componente, cómo viaja la información entre ellos, dónde está cada archivo y qué revisar cuando algo falla.

Está dirigido a un perfil técnico que tenga que mantener, defender o reconstruir el proyecto. Para el uso diario sin detalle interno está la *Guía de la publicación en la nube*; para operar el ambiente local está la *Guía completa del ambiente iShop*.

Todo lo que aparece aquí corresponde al sistema realmente desplegado y fue verificado contra el servidor en funcionamiento, no a un diseño previsto.

Arquitectura

El sistema son ocho contenedores Docker en un solo servidor. Cada uno tiene una responsabilidad única y se comunican por una red interna privada.

Componente	Imagen	Función
Caddy	`caddy`	Única puerta al exterior: TLS y enrutado por dominio
Tienda	archivos estáticos	HTML, CSS, JS del catálogo, servidos por Caddy
Chatwoot	`chatwoot/chatwoot`	Widget de chat, bandeja de asesores, API
Chatwoot Sidekiq	`chatwoot/chatwoot`	Trabajos en segundo plano de Chatwoot
PostgreSQL	`postgres`	Conversaciones, contactos, usuarios
Redis	`redis`	Cola de trabajos y canal de tiempo real
n8n	`n8nio/n8n`	Orquestador: contiene toda la lógica del asistente
Qdrant	`qdrant/qdrant`	Base de datos vectorial con el conocimiento
Ollama	`ollama/ollama`	Genera los vectores de texto (embeddings)

Fuera del servidor hay un único servicio externo: **Google Gemini**, que interpreta la intención del usuario y redacta la respuesta final.

La separación importante es esta: **Gemini no conoce el catálogo**. El catálogo vive en Qdrant, dentro del servidor. Gemini solo recibe, en cada consulta, los fragmentos que Qdrant seleccionó. Eso es lo que significa RAG y es lo que evita que el asistente invente productos o precios.

Topología de red

Solo los puertos 22, 80 y 443 están abiertos en el cortafuegos del proveedor. Todos los servicios internos escuchan **únicamente en `127.0.0.1`**, de modo que no son alcanzables desde Internet aunque alguien conozca la dirección IP:

```
n8n:      "127.0.0.1:5678:5678"
qdrant:   "127.0.0.1:6333:6333"
ollama:   "127.0.0.1:11434:11434"
chatwoot: "127.0.0.1:3000:3000"
```

Esa restricción se aplica con `compose.prod.yml`, que usa la etiqueta `!override` de Docker Compose. Sin esa etiqueta, Compose **fusiona** las listas de puertos en lugar de reemplazarlas, y los puertos públicos del archivo de desarrollo seguirían abiertos. Requiere Compose 2.24 o superior.

Caddy es el único proceso con puertos públicos y enruta por nombre de dominio:

Dominio	Destino
`100-52-28-14.sslip.io`	Archivos estáticos de la tienda
`chat.100-52-28-14.sslip.io`	`chatwoot:3000`
`n8n.100-52-28-14.sslip.io`	`n8n:5678`

Además bloquea rutas sensibles del repositorio que está montado como raíz web:

```
@privado {
    path /chatbot/*
    not path /chatbot/widget/*
}
respond @privado 404

@oculto path /.git/* /.github/* *.py *.sh *.yml *.pyc /.env*
respond @oculto 404
```

Se comprueba pidiendo `https://100-52-28-14.sslip.io/chatbot/.env`: debe responder 404.

Los certificados son de Let's Encrypt y Caddy los renueva solo. No hay nada que hacer manualmente.

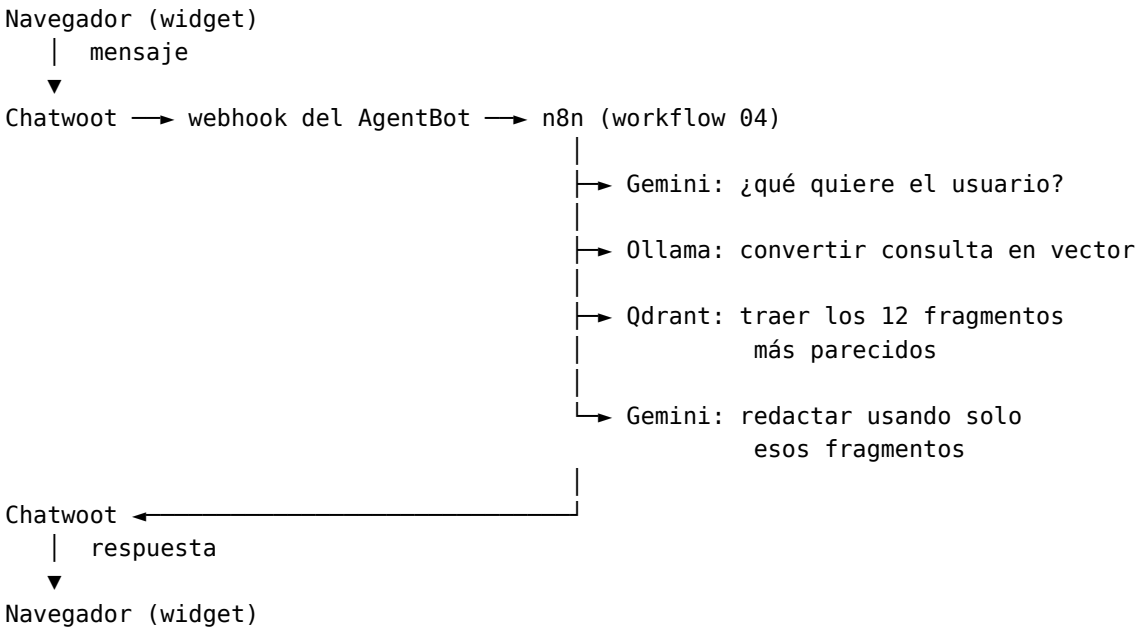
Estructura del repositorio

```
ishop/
├── index.html, products/, collections/, css/, js/   tienda estática
├── chatbot/
│   ├── compose.yml              servicios (desarrollo)
│   ├── compose.prod.yml         capa de producción
│   ├── Caddyfile                enrutado y TLS
│   ├── workflows/*.json         lógica del asistente
│   ├── knowledge/*.md           fuente del conocimiento RAG
│   ├── widget/                  integración del chat en la tienda
│   ├── scripts/*.py             utilidades de operación
```

```
│   └── deploy/                  scripts y guía de despliegue
├── GUIA-PAUL-ISHOP.md           operación del ambiente local
├── GUIA-NUBE-ISHOP.md           explicación de la publicación
└── MANUAL-TECNICO-ISHOP.md      este documento
```

Recorrido completo de una consulta

Este es el núcleo del proyecto. Cuando un visitante escribe en el widget, ocurre lo siguiente:



El workflow `04-chatwoot-agent-bot.json` tiene 21 nodos. Los relevantes, en orden de ejecución:

1. Webhook Chatwoot. Recibe el evento. Chatwoot envía todos los eventos de la conversación, no solo los mensajes del cliente.

2. Preparar evento. Normaliza el evento y decide si corresponde actuar. Descarta mensajes salientes, notas privadas y mensajes del propio bot; si no filtrara, el asistente se respondería a sí mismo en bucle. Aquí también detecta si el usuario pide un humano, comparando el texto contra un patrón que incluye *asesor, soporte, persona, humano, agente, reclamo* y *queja*.

3. ¿Debe responder la IA? Corta la ejecución si el paso anterior marcó que no corresponde.

4. ¿Pide un asesor? Si detectó petición de humano, deriva al traspaso (descrito más abajo) y la IA no responde.

5. Marcar escribiendo. Envía el indicador de "escribiendo" a Chatwoot para que la espera no parezca una caída.

6. Preparar petición Gemini / Gemini - Entender petición. Primera llamada al modelo. No responde al usuario: clasifica la intención y extrae la categoría de producto y el presupuesto si existe.

7. Interpretar intención. Toma la salida de Gemini y, si falla o devuelve algo inesperado, aplica una heurística local de respaldo. Esa heurística además corrige errores de escritura frecuentes (`microfno` → micrófono, `relok` → reloj Apple Watch, `mackbook` → MacBook) y arma dos cosas: el texto que se buscará en Qdrant y un filtro por categoría.

8. Buscar evidencia en Qdrant. Busca los 12 fragmentos más cercanos (`topK: 12`) en la colección `ishop_rag_v2`, aplicando el filtro de categoría para que una consulta sobre relojes no traiga portátiles.

9. Procesar evidencia RAG. Ordena los resultados, descarta los que no superan el umbral de similitud y arma el bloque de contexto. Si no hay evidencia suficiente, responde explícitamente que no la encontró y ofrece un asesor, en lugar de dejar que el modelo improvise.

10. Gemini - Redactar con evidencia. Segunda llamada al modelo, esta vez con el contexto recuperado y la instrucción de ceñirse a él.

11. Elegir respuesta final / Publicar en Chatwoot. Publica la respuesta en la conversación mediante la API de Chatwoot.

Hay además un atajo: para saludos y frases triviales, el nodo *¿Respuesta inmediata?* responde con un texto fijo sin gastar llamadas al modelo ni búsquedas vectoriales.

El pipeline RAG

Fuente del conocimiento

`chatbot/knowledge/` contiene los documentos que alimentan la base vectorial:

Archivo	Contenido
`catalogo-productos.md`	Catálogo completo con precios, variantes y URLs
`contenido-publico-ishop.md`	Envíos, garantía, pagos, políticas
`reglas-asistente.md`	Tono y límites de lo que el asistente puede decir
`preguntas-prueba.md`	Casos usados por la prueba automática

`sync_knowledge.py` regenera esos documentos leyendo la copia local de la tienda, de modo que el conocimiento no se mantiene a mano.

Indexación

`index_knowledge.py` hace el trabajo pesado:

- **Unidad de indexación: un producto, un documento.** No se parte el texto en trozos de tamaño fijo. Cada producto es un documento completo con su título, descripción, tipo, marca, etiquetas, precio mínimo, variantes y URL. Así una búsqueda nunca devuelve media ficha ni mezcla dos productos.

- **Embeddings:** modelo `nomic-embed-text:latest` servido por Ollama, que produce vectores de **768 dimensiones**.
- **Colección:** `ishop_rag_v2`, distancia **coseno**, recreada desde cero en cada indexación para evitar residuos de catálogos anteriores.
- **Lotes de 24 documentos**, con `keep_alive: -1` para que Ollama no descargue el modelo de memoria entre lotes.

Estado actual verificado: **218 puntos** del catálogo.

Documentos cargados desde el formulario

El workflow 01 publica un formulario en `https://n8n.TU-DOMINIO/form/cargar-conocimiento` que permite subir un PDF o un archivo de texto sin acceso al servidor. El flujo extrae el texto, lo parte por párrafos en fragmentos de unos 900 caracteres, calcula los vectores con el mismo modelo que el indexador y los escribe en la misma colección.

Esos puntos llevan `origin: carga_manual`, y por eso `index_knowledge.py` ya no borra la colección completa: ahora elimina solo los puntos cuyo `origin` no sea ese valor, de modo que una actualización del catálogo respeta lo que cargó un asesor. Los identificadores de los documentos subidos empiezan en mil millones para no chocar con los correlativos del catálogo.

Si la búsqueda de productos devuelve evidencia floja (puntaje máximo por debajo de 0.70) y el mensaje está redactado como pregunta, el nodo *Procesar evidencia RAG* repite la búsqueda sobre los documentos informativos. Esto existe porque el clasificador de intención conoce una lista fija de temas y no puede anticipar los de un documento recién subido.

El payload de cada punto conserva los campos estructurados, lo que permite filtrar antes de medir similitud:

```
doc_kind, category, title, url, product_type, brand,
tags, min_price, variants, description, content
```

Ese filtrado previo es lo que hace que "quiero un reloj" no devuelva un MacBook por parecido de redacción.

El traspaso a un asesor humano

Cuando *Preparar evento* detecta la petición de un humano, el nodo *Publicar en Chatwoot* ejecuta cuatro acciones:

1. Cambia el estado de la conversación a **abierta** (`toggle_status`).
2. Consulta `/api/v1/accounts/{id}/agents` y elige al asesor: primero el que coincida con `CHATWOOT_HUMAN_AGENT_EMAIL`; si no está definido, el primer administrador; y como último recurso, el primer agente.
3. Asigna la conversación con `POST /assignments`.
4. Deja una nota interna visible solo para el equipo.

A partir de ese momento el filtro del paso 2 impide que la IA siga respondiendo, porque la conversación ya no está asignada al bot. Para devolverla al asistente, el asesor cambia el estado a **pendiente**.

Configuración

Todo lo variable vive en `chatbot/.env`, que está fuera del control de versiones.

Variable	Para qué sirve
`SITE_DOMAIN`, `CHAT_DOMAIN`, `N8N_DOMAIN`	Dominios que atiende Caddy
`ACME_EMAIL`	Contacto para los certificados Let's Encrypt
`CHATWOOT_SECRET_KEY_BASE`	Firma de sesiones de Chatwoot
`CHATWOOT_POSTGRES_PASSWORD`	Acceso a la base de datos
`CHATWOOT_ACCOUNT_ID`	Cuenta sobre la que opera el bot
`CHATWOOT_ACCESS_TOKEN`	Token de administrador, usado por los scripts
`CHATWOOT_BOT_TOKEN`	Token del AgentBot, usado por n8n
`CHATWOOT_HUMAN_AGENT_EMAIL`	Asesor que recibe los traspasos
`GEMINI_API_KEY`	Acceso a Gemini
`ENABLE_ACCOUNT_SIGNUP`	Registro público de Chatwoot, en `false`
`COMPOSE_FILE`	Encadena `compose.yml:compose.prod.yml`

`COMPOSE_FILE` dentro del `.env` es lo que hace que **cualquier** `docker compose` ejecutado en esa carpeta aplique la configuración de producción sin tener que recordar banderas.

Detalle operativo: `docker compose restart` **no** vuelve a leer el `.env`. Para aplicar un cambio de variables hay que usar `docker compose up -d`, que recrea el contenedor.

Scripts

Script	Qué hace
`sync_knowledge.py`	Regenera los documentos de `knowledge/` desde la tienda
`index_knowledge.py`	Calcula embeddings y reconstruye la colección Qdrant
`setup_chatwoot.py`	Crea el inbox web y el AgentBot, y escribe la configuración del widget
`install_widget.py`	Inserta el widget en todas las páginas HTML
`sync_prompt_to_workflow.py`	Lleva el prompt editable a los workflows
`test_chatbot_e2e.py`	Prueba de extremo a extremo con 13 casos reales
`deploy/setup-server.sh`	Docker, Compose y swap en un servidor nuevo
`deploy/configure.sh`	Genera el `.env` conservando los secretos existentes
`deploy/start-prod.sh`	Levanta el stack en producción

Operación

Comandos frecuentes

```
ssh -i ~/.ssh/ishop-lightsail.pem ubuntu@100.52.28.14
cd ~/ishop/chatbot

docker compose ps                  # estado de los ocho servicios
docker compose logs -f n8n         # seguir la ejecución del asistente
docker compose logs -f caddy       # certificados y enrutado
docker compose up -d n8n           # recrear tras cambiar el .env
free -h                            # memoria disponible
```

Actualizar el catálogo

```
cd ~/ishop
python3 chatbot/scripts/sync_knowledge.py
python3 chatbot/scripts/index_knowledge.py
```

Respaldo

```
cd ~/ishop/chatbot
docker compose exec -T chatwoot-postgres \
  pg_dump -U postgres chatwoot_production > ~/respaldo-chatwoot.sql
```

El conocimiento vectorial no necesita respaldo: se reconstruye con `index_knowledge.py` a partir de los documentos del repositorio.

Prueba automática

```
cd ~/ishop
python3 chatbot/scripts/test_chatbot_e2e.py
```

Recorre 13 casos (consultas por categoría, comparaciones, presupuesto, preguntas de políticas y un traspaso a humano) contra la API real de Chatwoot. Resultado de la última ejecución: **13/13 correctas, 1.02 s de promedio**.

Fallas conocidas y cómo diagnosticarlas

Estas situaciones ya ocurrieron durante el despliegue. Se documentan porque ninguna da un mensaje de error que apunte a la causa.

El bot no responde y n8n registra "webhook not registered"

Activar un workflow con `n8n update:workflow --active=true` marca la bandera en la base de datos pero **no registra el webhook**, ni siquiera reiniciando el contenedor. Hay que activarlo por la API, que sí lo registra:

```
curl -b cookie.txt -X POST \
  "https://n8n.DOMINIO/rest/workflows/$ID/activate" \
  -H "Content-Type: application/json" -d "{\"versionId\":\"$VER\"}"
```

Se comprueba enviando un ping al webhook: debe responder 200, no 404.

El bot responde vacío y el log dice "Invalid Qdrant URL"

El campo de la credencial de Qdrant en n8n se llama **`qdrantUrl`**, no `url`. Con cualquier otro nombre, n8n guarda la credencial vacía sin avisar y el nodo falla en ejecución. El archivo `deploy/n8n-credentials.json` ya trae el nombre correcto.

Se duplican los workflows al importarlos

`n8n import:workflow` crea copias nuevas si el JSON no trae un campo `id` estable. El efecto peligroso es terminar con dos workflows activos compitiendo por el mismo webhook, con resultados impredecibles. Los cuatro archivos del repositorio ya llevan su `id` fijo.

Para borrar un workflow sobrante hay que archivarlo primero; el borrado directo devuelve 400:

```
curl -b cookie.txt -X POST "https://n8n.DOMINIO/rest/workflows/$ID/archive"
curl -b cookie.txt -X DELETE "https://n8n.DOMINIO/rest/workflows/$ID"
```

Un asesor entra a Chatwoot y no ve ninguna conversación

La pantalla de configuración inicial de Chatwoot **crea una cuenta nueva cada vez que se completa**. Un asesor que la use en lugar de iniciar sesión termina en una cuenta vacía, sin inbox ni conversaciones, y nada indica que está en el lugar equivocado. Los asesores adicionales se agregan desde **Settings → Agents** de la cuenta existente.

Para verificar en qué cuenta está cada usuario:

```
select au.account_id, a.name, u.email, au.role
from account_users au
join users u on u.id = au.user_id
join accounts a on a.id = au.account_id;
```

Un usuario de n8n no ve los workflows

En la edición gratuita de n8n, compartir workflows entre usuarios es función de pago. La alternativa usada aquí fue **transferir** los workflows al proyecto personal del asesor, con lo que ambos usuarios los ven.

Advertencia importante: al transferir un workflow, **las credenciales no se mueven con él** y el workflow deja de funcionar sin ningún error visible en la interfaz. Hay que transferir también las credenciales y volver a activar el workflow.

La indexación tarda más de una hora

El servidor tiene procesador por ráfagas: acumula créditos cuando está ocioso y los gasta cuando trabaja. Al agotarlos, el rendimiento cae drásticamente. Para una reindexación completa conviene calcular los embeddings en una computadora personal y escribir en el Qdrant remoto a través de un túnel SSH:

```
ssh -i ~/.ssh/ishop-lightsail.pem -L 6400:localhost:6333 ubuntu@100.52.28.14
QDRANT_URL=http://localhost:6400 python3 chatbot/scripts/index_knowledge.py
```

Eso baja el proceso de más de una hora a unos cinco minutos.

Decisiones de diseño y límites

Por qué n8n y no código. La lógica del asistente queda visible como un diagrama y se puede modificar sin recompilar ni desplegar. Para un proyecto que debe explicarse y defenderse, eso vale más que la eficiencia de un servicio escrito a mano.

Por qué embeddings locales y modelo de lenguaje externo. Los embeddings se calculan miles de veces y no requieren un modelo grande: hacerlos locales con Ollama elimina ese costo recurrente. La redacción sí requiere un modelo potente, y mantener uno así en un servidor pequeño no es viable.

Límites conocidos:

- Todo corre en un solo servidor. No hay alta disponibilidad: si la máquina cae, el sistema cae.
- La memoria está ajustada (2.4 GB usados de 3.7 GB, con 2 GB de swap). No conviene añadir servicios sin ampliar el plan.
- El dominio es de un servicio gratuito basado en la dirección IP. Si la IP cambia, cambia el dominio.
- El asistente depende de un servicio externo para redactar; si Gemini no responde, el sistema avisa y ofrece un asesor humano, pero no redacta solo.
- La base vectorial se reconstruye entera en cada actualización; no hay indexación incremental.